

Guida strutturata alle Prove Pratiche — Laboratorio di Sistemi Operativi

Questo documento **non** contiene le soluzioni complete: fornisce, per ogni prova contenuta nello zip, una **struttura ragionata** (quali system call usare, quale schema di codice seguire, quali insidie evitare), agganciata alla teoria dei due PDF (la dispensa di Luca Argentino e gli appunti delle esercitazioni).

Le prove sono prima raggruppate per **tema ricorrente** (è il modo più utile per prepararsi, perché gli stessi schemi tornano ogni anno), poi elencate una per una in ordine cronologico con la traccia di risoluzione.

0. Lo scheletro comune a TUTTE le prove

Ogni prova ha la stessa struttura: Esercizio 0 (setup), Esercizio 1 in C (obbligatorio, 20 pt), Esercizio 2 in C (estensione, 10 pt), Esercizio 3 in Python/bash (10 pt), Esercizio 4 (consegna). Conviene memorizzare i pezzi fissi.

Esercizio 0 — sempre identico

Rendere la home inaccessibile ad altri (niente lettura né esecuzione) e tenere una sola directory in `/public` col proprio username e permessi `700`:

```
chmod go-rx ~
mkdir -p /public/nome.cognome
chmod 700 /public/nome.cognome
```

Collegamento alla teoria: tabella permessi negli appunti esercitazioni — `r=4 w=2 x=1`, `chmod`, e il fatto che su una directory `x` significa "attraversabile". Togliere `r` e `x` agli altri è esattamente `go-rx`.

Vincoli da rispettare ovunque (pena esercizio non valido)

- **Mai** `system()`, `popen()`, né `exec` di `"sh -c ..."`. In Python: `subprocess`, mai `os.system` / `os.spawn`.
- Tradotto: ogni volta che vuoi "lanciare un comando" devi farlo **a mano** con lo schema `fork()` + `exec*()` + `wait()` (dispensa §2.8.2 "mini-shell" e §2.3-2.6).
- Negli anni recenti: niente commenti nei file consegnati; allegare `sha1sum` dei file; rinominare i `.py`.

Lo schema "esegui un programma" (vale per metà delle prove)

Questo è IL pattern che devi avere nelle dita. Sta tutto nella dispensa §2.8.2 e negli appunti ("embrione della shell"):

```
pid_t pid = fork();
if (pid == 0) { // figlio
    // eventuali dup2() per ridirigere stdin/stdout/stderr
    execvp(file, argv); // o execv / execve / execlp
    perror("exec"); _exit(127); // raggiunta SOLO se exec fallisce
}
// padre
int status;
waitpid(pid, &status, 0);
// WIFEXITED(status), WEXITSTATUS(status), WIFSIGNALED(status)
```

Teoria di supporto: valori di ritorno di `fork()` (dispensa §2.3), `execve` non ritorna mai se ha successo (§2.8), `wait` / `WEXITSTATUS` (§2.6), gestione errori con `errno` / `perror` (§2.9).

Lo schema "visita ricorsiva di un albero" (l'altra metà delle prove)

Tantissime tracce chiedono di scorrere un sottoalbero del file system. Due strade:

1. `nftw()` (la più rapida da scrivere) — una callback per ogni nodo.
2. **Ricorsione manuale** con `opendir` / `readdir` / `closedir` + `lstat` (dispensa §3.4) e, per non concatenare stringhe / non fare `chdir`, `openat` / `fdopendir` (richiesto esplicitamente in 2022.07.22 es.2).

Scheletro ricorsivo manuale:

```
void walk(const char *path) {
    DIR *d = opendir(path);
    struct dirent *de;
    while ((de = readdir(d)) != NULL) {
        if (!strcmp(de->d_name, ".") || !strcmp(de->d_name, "..")) continue;
        char child[PATH_MAX];
        snprintf(child, sizeof child, "%s/%s", path, de->d_name);
        struct stat st;
        lstat(child, &st); // lstat: NON segue i symlink
        if (S_ISDIR(st.st_mode)) walk(child);
        else { /* elabora il file */ }
    }
    closedir(d);
}
```

Punto teorico cruciale (dispensa §3.3 e appunti): **stat segue** i link simbolici, **lstat no**. Quasi tutte le prove sui link richiedono **lstat** per *vedere* il link invece del bersaglio.

1. Mappa per temi (come studiare in modo efficiente)

Tema	Prove che lo usano	Teoria nei PDF
Link fisici / simbolici (trovare, invertire, convertire, confrontare inode)	2020.01.17, 2020.02.21, 2022.01.18, 2022.02.17, 2022.06.08, 2023.05.24, 2023.06.14, 2023.09.14, 2024.07.23(t1), 2025.01.21, 2025.02.13, 2025.05.29, 2026.01.12	§3.2-3.4 stat/lstat/dirent; appunti: tabella link / symlink / readlink , inode
Visita ricorsiva di alberi (tree, copia, confronto, catalogo)	2022.07.22, 2023.01.19(py), 2023.07.20, 2024.01.19(py), 2024.02.14, 2025.07.23, 2025.09.04	§3.4 readdir/scandir; openat/fdopendir
fork+exec+wait / rilancio / clone processi	2022.09.07, 2023.05.24(es2), 2024.01.19, 2024.02.14, 2024.05.30, 2024.06.24	§2.3-2.9 mini-shell, errno; appunti /proc
Pipe e ridirezione (ls sort , stdin2pipe)	2023.01.19	§4.4 dup2, §4.6 pipe
Segnali (trasferire dati via segnali, inviare segnali)	2022.06.22, 2023.01.23(es2), 2025.06.24, 2026.02.09	§5 kill/signal/sigaction, sigqueue/SA_SIGINFO
I/O su file e copia (inverti, copia parallela, lseek/buchi)	2023.02.16, 2024.07.23(t2)	§4.1-4.3 fd, read/write, lseek, sparse file
inotify (reagire a file creati in una directory)	2021.06.24, 2021.09.16, 2024.06.24	NON nei PDF → vedi §3 sotto
timerfd / poll / pidfd (timer, timeout)	2023.06.14, 2024.09.11	poll in §6; timerfd/pidfd NON nei PDF → §3
FIFO / named pipe	2023.01.23, 2024.01.19(es2)	appunti mkfifo / mknod ; pipe §4.6
dlopen / librerie dinamiche	2021.07.15	NON nei PDF → §3
/proc (info processi)	2022.06.08(py), 2023.05.24, 2024.05.30, 2024.06.24(py)	appunti: pid/ppid, ps ; /proc accennato
sha1 / hashing	2024.07.23(t1 py), 2025.07.23, 2025.09.04, 2026.01.12(consegna)	non concettuale: usare libreria o comando

Lezione strategica

L'**Esercizio 1** obbligatorio è quasi sempre uno di tre archetipi: visita di un albero con **lstat** , oppure **fork+exec+wait** , oppure segnali/pipe/timer. Se padroneggi i tre scheletri del §0, hai in mano il 70% di ogni prova. L'**Esercizio 3** (Python/bash) è quasi sempre una variante "scripting" dello stesso tema dell'es.1 (trovare file, ordinarli per data/inode/profondità, filtrarli per contenuto), risolvibile con **find** , **stat** , **sort** , **shasum** , **grep** .

2. Argomenti delle prove che NON sono coperti dai due PDF

Importante saperlo per non perdere tempo a cercarli nella dispensa. Per questi serve il **man** durante la prova (è consentito) o studio extra:

- inotify** (**inotify_init1** , **inotify_add_watch** , lettura di **struct inotify_event**): usato in 2021.06.24, 2021.09.16, 2024.06.24. È un fd su cui fai **read()** bloccante e ricevi eventi quando in una directory compaiono file (**IN_CLOSE_WRITE** / **IN_MOVED_TO**).
- timerfd_create** (timer come file descriptor su cui leggere le scadenze): 2023.06.14, e combinato con **poll** in 2024.09.11.
- pidfd_open** (un fd che diventa "pronto" quando un processo muore, da usare in **poll**): 2024.09.11.
- dlopen / dlsym** (caricare una libreria dinamica a runtime e invocarne una funzione): 2021.07.15.
- getopt** (parsing di opzioni **-j** , **-s** , **-l** , **-p**): 2023.02.16, 2025.02.13.
- realpath(3)** (calcolare il path assoluto canonico): 2020.02.21, 2022.06.08.
- sigqueue + SA_SIGINFO** (segnali che portano un dato, **si_value** / **si_int**): 2022.06.22, 2026.02.09. Qui la dispensa §5.5 aiuta (struttura **siginfo_t** , **SA_SIGINFO**), ma non parla di **sigqueue** lato mittente.

- `/proc` come fonte di dati sui processi (`/proc/PID/cmdline` , `/exe` , `/status`): appena accennato negli appunti, da approfondire.

3. Le 32 prove, una per una

Per ognuna: tema, system call chiave, schema risolutivo, riferimento teorico.

2020.01.17 — `searchlink` (link fisici e simbolici)

- **Es.1**: dati un file `f` e una directory `d` , elencare tutti i path nell'albero di `d` che si riferiscono a `f` come link fisico o simbolico.
- **Schema**: prendi l'inode di `f` con `stat(f) → (st_dev, st_ino)` . Visita ricorsiva di `d` (scheletro `$0`): per ogni voce fai `lstat` . Se è regolare con stesso `st_ino / st_dev → hard link` . Se è symlink (`S_ISLNK`), risolvi con `readlink / realpath` e se punta a `f → symlink` .
- **Es.2**: opzioni `-c / -l / -s` per sostituire i link trovati con copia / hard link / symlink. Userai `unlink + (link / symlink / copia con read/write §4.2.1)` .
- **Teoria**: §3.2-3.4, appunti tabella inode/link.

2020.02.21 — `abslink` / `absls` (realpath)

- **Es.1**: sostituire un symlink con uno equivalente che punti al **path assoluto**. `readlink` per leggere il target, `realpath` per renderlo assoluto, `unlink + symlink` .
- **Es.2**: `absls` stampa per ogni file di una dir il path completo (per i symlink, il path completo del puntato → `realpath`).
- **Es.3 (script)**: listato di una dir ordinato per **suffisso** → `ls + sort` con chiave sull'estensione, oppure Python con `sorted(key=...)` .
- **Nota**: `realpath` non è nei PDF.

2021.06.24 — `dircat` (inotify)

- **Es.1**: directory vuota `D` + file `F` ; ogni volta che un file appare in `D` , appendi a `F` una riga di intestazione col nome + il contenuto del file, poi cancellalo.
- **Schema**: `inotify_init1` , `inotify_add_watch(D, IN_CLOSE_WRITE|IN_MOVED_TO)` , loop con `read()` sugli eventi; per ogni evento apri il file (`open + read` , §4.2), scrivi su `F` (`open` con `O_APPEND`), poi `unlink` .
- **Es.2**: se il file è un **eseguibile**, invece del contenuto inserisci in `F` l'**output dell'esecuzione** → `fork + exec` con `dup2` di stdout su `F` (§4.4).
- **Nota**: inotify NON nei PDF (vedi §2).

2021.07.15 — `lancia` / `autolancia` (dlopen)

- **Es.1**: caricare `hw.so` a runtime e invocarne il `main` . `dlopen(path_assoluto, RTLD_NOW)` , `dlsym(h, "main")` , cast a puntatore a funzione, chiamala con `argc/argv`. Linkare con `-ldl` .
- **Es.2**: `autolancia` riconosce se `argv[1]` è libreria dinamica o eseguibile ELF (leggi i primi byte / header) e gestisce entrambi (dlopen vs fork+exec).
- **Es.3 (script)**: trovare nel sottoalbero il file modificato più di recente e quello più vecchio → `find ... -printf '%T@ %p\n' | sort -n` (primo e ultimo).
- **Nota**: dlopen NON nei PDF.

2021.09.16 — `execname` (inotify + exec)

- **Es.1**: crea dir `exec` ; quando vi compare un file, **interpreta il nome del file come comando con parametri**, eseguillo, cancellalo. (`touch 'exec/echo ciao mare'` → stampa `ciao mare` .) inotify per rilevare + split del nome in `argv` + `fork / execvp` .
- **Es.2**: `execname2` scrive l'**output** nel file stesso invece di cancellarlo → `dup2` di stdout sul file (§4.4).
- **Es.3 (script)**: lista ricorsiva con, per ogni nome, in quali sottodirectory compare; ordinato alfabeticamente → Python con dict `nome -> [dir...]` .

2022.01.18 — `abssymlink` / `cpsymlink`

- Variante di 2020.02.21 ma su **tutta una directory**. Es.1: trasforma ogni symlink in uno assoluto (`readlink + realpath + unlink + symlink`). Es.2: sostituisci ogni symlink che punta a file regolare con la **copia** del puntato (read/write §4.2.1).
- **Es.3 (script)**: file che contengono un pattern, ordinati dal più recente al più vecchio → `grep -rl pattern | xargs stat ... | sort` .

2022.02.17 — `nonest_symlink` / `nest2hard`

- Symlink **nidificati** (symlink → symlink). Es.1: cancellali. Es.2: sostituiscili con un hard link al file finale puntato. Per rilevare la nidificazione: `readlink` del target e `lstat` del target → se anch'esso è `S_ISLNK` , è nidificato.
- **Es.3 (script)**: identico a 2022.01.18 es.3 (pattern + ordinamento per data).

2022.06.08 — `invsymlink` (inversione link)

- **Es.1**: dato un symlink `B → A` , scambia: `B` diventa il file reale, `A` diventa un symlink a `B` . Usa `realpath` per i path completi, poi `rename / unlink / symlink` . Attenzione all'atomicità (la traccia consiglia `realpath(3)`).
- **Es.2**: estendi a tutti i symlink di una directory.

- **Es.3 (script):** tabella processi dell'utente da `/proc/*/status` con nome eseguibile e `VmSize` → Python che legge `/proc/<pid>/status`.

2022.06.22 — tx / rx (segnali con sigqueue)

- **Es.1:** trasferire stringhe ≤8 caratteri usando il **valore** allegato al segnale (`sigqueue`, parametro `value`). `rx` stampa il proprio pid e attende; `tx pid msg` impacchetta gli 8 byte nell'intero `sival_int / sival_ptr` e fa `sigqueue`.
- **Schema lato rx:** `sigaction` con `SA_SIGINFO`, handler `(int, siginfo_t*, void*)`, leggi `si->si_value` e `si->si_pid` (dispensa §5.5).
- **Es.2:** stringhe di lunghezza arbitraria → iterare 8 byte alla volta.
- **Es.3 (script):** catalogo file per output di `file` (categoria → elenco).
- **Teoria:** §5.5 SA_SIGINFO/siginfo_t; `sigqueue` da `man`.

2022.07.22 — tree (openat/fdopendir)

- **Es.1:** stampa il sottoalbero (stile comando `tree`).
- **Es.2 (alternativo, 30 pt): una sola funzione ricorsiva**, senza concatenare stringhe né `chdir`, usando `openat` + `fdopendir`. Questo è il modo "pulito" di ricorrere: passi il fd della dir corrente, `openat(dirfd, name, ...)` per scendere.
- **Es.3 (script):** esegui in sequenza tutti gli **script** di una dir ma non i binari ELF → distingui con il comando `file` o leggendo il magic `\x7fELF`.
- **Teoria:** §3.4 dirent; `openat/fdopendir` da `man`.

2022.09.07 — rilancia (fork+exec+wait+tempo)

- **Es.1:** esegui `argv[1]` con i suoi parametri; se termina **senza errori** (non per segnale e con exit 0) riesegui. Loop con `fork / execvp / waitpid` + `WIFEXITED / WEXITSTATUS / WIFSIGNALED` (§2.6).
- **Es.2:** se l'esecuzione dura **meno di un secondo**, non rilanciare. Misura il tempo prima/dopo con `clock_gettime / time`.
- **Es.3 (script):** somma in byte degli eseguibili ELF nelle dir passate → `find` + `file` + somma di `stat -c%s`.

2023.01.19 — stdin2pipe (pipe + dup2)

- **Es.1:** leggi due righe (due comandi); collega l'output del primo all'input del secondo (come `cmd1 | cmd2`). Questo è **esattamente** l'esempio della dispensa §4.6.2 (`ls | sort -r`): `pipe()`, due `fork`, nel primo `dup2(fd[1], STDOUT)`, nel secondo `dup2(fd[0], STDIN)`, il padre chiude entrambi i capi e fa due `wait`.
- **Es.2:** N righe → catena di N-1 pipe. Generalizza con un array di pipe in un loop.
- **Es.3 (script):** merge di due alberi in un terzo, concatenando i file omonimi.
- **Teoria:** §4.4 dup2, §4.6 pipe — è la prova più "da manuale".

2023.01.23 — fifotext / fifosig (FIFO + segnali)

- **Es.1:** crea una FIFO (`mkfifo`), aprila in lettura, stampa ogni riga; se la pipe viene chiusa **riapri**; alla riga `FINE` termina e cancella la FIFO.
- **Schema:** `mkfifo(path, 0666)`, loop esterno di `open(O_RDONLY)`, loop interno di lettura riga per riga; quando `read` ritorna 0 (EOF, scrittore chiuso) richiudi e riapri. (Teoria: FIFO negli appunti `mknod / mkfifo`; semantica EOF della pipe §4.6.)
- **Es.2:** `fifosig` legge righe `pid segnale` e invia il segnale (`kill`, §5.2).
- **Es.3 (script):** `difdir` → due dir, crea due nuove dir con i soli file dal nome in comune, copiandone le rispettive versioni.

2023.02.16 — pcp (copia parallela)

- **Es.1:** copia parallela: due figli, uno copia la prima metà, l'altro la seconda. Calcola la dimensione con `stat / lseek(..,SEEK_END)`, ogni figlio fa `lseek` al proprio offset e copia il proprio blocco (`pread / pwrite` sono comodi: offset esplicito e atomici — appunti). Padre fa due `wait`.
- **Es.2:** grado di parallelismo con `getopt` (`-j 8` → 8 figli, ciascuno 1/8).
- **Es.3 (script):** `ccpl` conta i caratteri delle righe corrispondenti di tutti i file (somma dei caratteri della 1ª riga di ogni file, della 2ª, ...).
- **Teoria:** §4.2–4.3 read/write/lseek, sparse file; `getopt` da `man`.

2023.05.24 — pidcmd / clona processo (/proc)

- **Es.1:** stampa i PID dei processi lanciati con una specifica riga di comando → scorri `/proc/*/cmdline` (campi separati da `\0`) e confronta con `argv`.
- **Es.2:** dato un PID, lancia un processo identico (stesso exe, argv, environ, cwd) → leggi `/proc/PID/cmdline`, `/exe`, `/environ`, `/cwd`, poi `fork + execve`.
- **Es.3 (script):** cerca symlink che puntano allo stesso file (confronto **inode** del target) → `find -type l` + `readlink -f` + raggruppa per inode (`stat -c%i`).

2023.06.14 — tfdtest / mftdtest (timerfd)

- **Es.1:** stampa una stringa a intervalli regolari usando `timerfd_create`. Parametro `n,intervallo,stringa`; arma il timer con `timerfd_settime`, leggi le scadenze con `read` sul fd, stampa il timestamp.
- **Es.2:** più timer contemporaneamente → un fd timer per ciascuno, monitorati con `poll` (§6) o `epoll`.

- **Es.3 (script):** nel sottoalbero, cancella i symlink **relativi** e lascia gli assoluti → `readlink` e controlla se inizia con `/`.
- **Nota:** timerfd NON nei PDF; poll sì (§6.3).

2023.07.20 — `cprl` (copia albero con hard link)

- **Es.1:** come `cp -rl`: ricrea la struttura di directory di `a` in `b`, ma i file non vengono copiati bensì collegati con **hard link** (`link`). Ricorsione (`$0`) + `mkdir` per le dir + `link` per i file. Da fare **solo in C**, senza lanciare comandi.
- **Es.2:** `cprlt tempo a b` → i file più vecchi del tempo dato vengono hard-linkati, gli altri copiati (read/write). Confronta `st_mtime` con il parametro.
- **Es.3 (script):** elenca file/dir con nomi non rappresentabili in ASCII.

2023.09.14 — `vreaddir` (allocazione dinamica)

- **Es.1:** funzione `char **vreaddir(const char *path)` che ritorna i nomi dei file come vettore di stringhe terminato da `NULL` (formato argv/envp), tutto allocato dinamicamente. `opendir` / `readdir`, `malloc` / `realloc` del vettore, `strdup` dei nomi.
- **Es.2:** vettore + stringhe in **un unico malloc** (così basta un `free`). Calcola prima la dimensione totale (puntatori + byte dei nomi), poi un solo blocco e disponi i puntatori in testa e le stringhe in coda.
- **Es.3 (script):** trova nel sottoalbero i symlink che puntano allo stesso file (raggruppa per inode del target, come 2023.05.24 es.3).
- **Teoria:** §3.4 dirent; gestione memoria standard C.

2024.01.19 — `argsend` / `argrecv` / `pargrcv`

- **Es.1:** `argsend` concatena argv[1..] (col terminatore `\0`) e li scrive su stdout; `argrecv` legge quella sequenza da stdin, ricostruisce argv ed esegue il comando (`fork` + `execvp`). Parsing: spezza sui `\0`, costruisci `char *argv[]` terminato NULL.
- **Es.2:** `pargrcv` crea una **named pipe** e quando vi si ridireziona l'output di `argsend`, esegue il comando. (FIFO come in 2023.01.23.).
- **Es.3 (script):** `tcmp` confronta due alberi: path comuni, oppure solo-in-1 (`-1`), solo-in-2 (`-2`).

2024.02.14 — `search_name` / `run_name`

- **Es.1:** cerca nel sottoalbero gli **eseguibili** con un certo nome e indica per ciascuno se è **script** o **ELF** (leggi il magic: `#!` → script, `\x7fELF` → ELF). Visita ricorsiva (`$0`) + `access(path, X_OK)` per "eseguibile".
- **Es.2:** `run_name` li esegue uno dopo l'altro, ciascuno con la **cwd** pari alla dir in cui si trova → `fork`, nel figlio `chdir` + `execv`, padre `wait`.
- **Es.3 (script):** catalogo file per categoria di `file` (come 2022.06.22 es.3).

2024.05.30 — `cloneproc` (/proc)

- **Es.1:** dato un PID, esegue una copia (stesso file, stesso argv) leggendo `/proc/PID/exe` e `/proc/PID/cmdline`; `fork` + `execve`. Più programma di test.
- **Es.2:** `cloneproc+` clona anche **cwd** e **environment** (`/proc/PID/cwd`, `/proc/PID/envIRON`) → `chdir` + `execve` con l'envIRON ricostruito.
- **Es.3 (script):** `lscmd` elenca i PID dell'utente raggruppati per pathname del programma → `/proc/*/exe` con `readlink`.
- **Teoria:** §2.8 execve, §2.6 wait; `/proc` dagli appunti.

2024.06.24 — `inotirun` (inotify + esecuzione)

- **Es.1:** con **inotify**, sorveglia una dir `D` vuota; i file che vi compaiono hanno formato "pathname eseguibile + una riga per ogni argv", vengono eseguiti e cancellati. Leggi il file, costruisci argv dalle righe, `fork` + `execv`, poi `unlink`.
- **Es.2:** `inotimrun` → un file può contenere **più comandi** separati da riga vuota; esegui in sequenza.
- **Es.3 (script):** `permdir` crea una dir per ogni stringa di permessi, con symlink ai file aventi quei permessi → `stat -c%A` per raggruppare.
- **Nota:** inotify NON nei PDF.

2024.07.23 turno 1 — `directory ...` + symlink (rename atomico)

- **Es.1:** crea sottodir `...`; sposta lì tutti i file regolari e sostituiscili nella dir corrente con un **symlink relativo** alla nuova posizione, usando `rename` per l'atomicità (il file non deve mai sparire).
- **Es.2:** undo — rimpiazza i symlink che puntano a `../nomefile` con i veri file (`rename`).
- **Es.3 (script):** date due dir, cancella (da entrambe) i file con la **stessa sha1** presenti in entrambe → `shasum` + confronto insieme.

2024.07.23 turno 2 — `inverti` (I/O in place)

- **Es.1:** inverti i byte di un file **sul posto**, senza file temporanei, con buffer ≤ 2K. Due offset (inizio/fine) che convergono: `pread` / `pwrite` ai due estremi, scambia i blocchi. Attento alla lunghezza arbitraria e al blocco centrale.
- **Es.2:** `pinverti nprocessi pathname` → divide il file in N blocchi, N processi paralleli invertono i rispettivi estremi.
- **Es.3 (script):** come turno 1 es.1 ma in Python/bash (dir `...` + symlink relativi).
- **Teoria:** §4.2–4.3 read/write/lseek; `pread`/`pwrite` atomici (appunti).

2024.09.11 — `timeout` (poll + pidfd + timerfd)

- **Es.1:** esegui un programma e terminalo se supera una durata (ms). **Deve** usare `poll` + `pidfd_open` + `timerfd*`. Schema: `fork` + `exec`

del comando, `pidfd_open` sul figlio, `timerfd` armato al timeout, `poll` su entrambi: se scade prima il timer → `kill` del figlio; se diventa pronto il `pidfd` → il figlio è finito.

- **Es.2:** se il programma termina con **errore** prima del timeout, riattivalo allo scadere del timeout.
- **Es.3 (script):** `slinout` elenca i symlink del sottoalbero divisi in **interni** (puntano dentro l'albero) ed **esterni**; target assoluto o relativo da normalizzare.
- **Teoria:** poll §6.3; `pidfd/timerfd` da `man` (NON nei PDF).

2025.01.21 — `samecont` (dimensione/hard link)

- **Es.1:** dati file `f` e dir `d`, elenca i file nel sottoalbero di `d` che hanno la **stessa dimensione** di `f` ma **non** sono hard link di `f` (stesso `st_size`, diverso `st_ino`). Visita ricorsiva + `stat`.
- **Es.2:** elenca i **symlink** che puntano a `f` nel sottoalbero (`readlink/realpath`).
- **Es.3 (script):** lista dei file ordinati per **profondità** (prima i più profondi, per ultimi quelli della radice); a parità di livello, ordine crescente del nome.

2025.02.13 — `ckfile` (opzioni con `getopt`)

- **Es.1:** a seconda dell'opzione, elenca i symlink che puntano a `f` (`-s`) o gli hard link di `f` (`-l`) nel sottoalbero di `d`. Parsing con `getopt`.
- **Es.2:** altre opzioni: file con **stesso contenuto** di `f` (confronto byte a byte), e con `-p N` file che coincidono nei primi N byte.
- **Es.3 (script):** file con stessa dimensione di `f` ma non hard link (= 2025.01.21 es.1 in scripting).

2025.05.29 — `search_prec` / `search_drec` (file "ricorsivi")

- **Es.1:** "file path ricorsivo" = file il cui **contenuto** coincide col proprio pathname completo. Scorri la dir corrente, per ogni file confronta il contenuto con il suo path (costruito con `getcwd` + nome). `search_prec` senza parametri.
- **Es.2:** "file dir ricorsivo" = contenuto identico al pathname completo di **un** file della dir corrente. Confronta ogni contenuto con l'insieme dei pathname.
- **Es.3 (script):** a seconda del nome con cui è invocato (`cksymlink/cklink`), elenca i symlink o gli hard link di `f` in `d` (usa `argv[0]` / `$0`).

2025.06.24 — `semsend` / `semrecv` (segnali bit a bit)

- **Es.1:** trasferire una stringa **bit a bit** usando `SIGUSR1/SIGUSR2` (uno per il bit 0, l'altro per il bit 1). `semrecv` stampa il pid e attende; `semsend pid str` invia, per ogni carattere (terminatore incluso), 8 segnali. Lato rx: `sigaction`, ricomponi i byte bit a bit. Serve sincronizzazione (es. `pause/sigsuspend`).
- **Es.2:** `semrecv` riceve da **più mittenti** contemporaneamente (bit inframmezzati) → distingui per `si_pid` (`SA_SIGINFO`, §5.5), un buffer per mittente.
- **Es.3 (script):** aggiungi righe di commento in testa ai file `.c/.sh/.py` rispettando la sintassi di ciascun linguaggio (e il `#!` iniziale negli script).
- **Teoria:** §5.3 signal, §5.5 `sigaction/siginfo_t`.

2025.07.23 — `shaldir` / `shaldiff`

- **Es.1:** ricostruisci in una nuova dir l'albero della dir sorgente; ai file regolari corrispondono file che contengono la **sha1** dell'originale. Ricorsione (`$0`) + `mkdir` + calcolo sha1 (libreria, es. OpenSSL, o implementazione propria).
- **Es.2:** `shaldiff` confronta e mostra i file modificati (sha1 diversa).
- **Es.3 (script):** `dremcont f d` cancella tutti i file in `d` col contenuto uguale a `f` → confronto con `cmp` o per sha1.

2025.09.04 — `shalindex` / `shalupdate`

- **Es.1:** per ogni file regolare nella dir, crea in `.shalindex/` un **symlink** che punta al file e ha come nome la sha1 del contenuto.
- **Es.2:** `shalupdate` aggiorna: rimuove i link a file cancellati, ricalcola e ricrea i link per i file modificati dopo la creazione del link (`st_mtime` vs `mtime` del link con `lstat`).
- **Es.3 (script):** `inotab` elenca `inode pathname` di tutto il sottoalbero, ordinato per inode crescente, con parametro opzionale di profondità massima.

2026.01.12 — `modifcmp` (confronto tempi di modifica)

- **Es.1:** tre modalità in base agli argomenti — (a) un file: elenca i file del sottoalbero corrente **più recenti** di quello; (b) due file: stampa il secondo se più recente del primo; (c) file + dir: file del sottoalbero della dir più recenti del file. `stat` + confronto `st_mtim` (precisione ai nanosecondi se serve), visita `$0`.
- **Es.2:** `modif=` → file con tempo di modifica **uguale**, che non siano hard/sym link del file. Mostra anche come costruire l'esempio di test (`touch -r` per copiare `mtime`).
- **Es.3 (script):** copia un file UTF-8 sostituendo i caratteri **non ASCII** con `?` (byte $\geq 0x80$ → gestire le sequenze multibyte UTF-8 e mapparle a un singolo `?`).

2026.02.09 — `sigtx` / `sigrx` (segnali con `sigqueue`, 64bit)

- **Es.1:** identico nello spirito a 2022.06.22: trasferire stringhe ≤ 8 caratteri via `sigqueue` (il `value` su 64bit contiene gli 8 byte). `sigrx`

stampa il pid e attende; `sigtx pid msg` impacchetta e invia.

- **Es.2:** stringhe arbitrarie, 8 byte alla volta, **con ack**: il ricevente conferma (es. con un proprio segnale al mittente) prima del blocco successivo.
 - **Es.3 (script):** file che contengono un pattern ASCII, ordinati dal più **vecchio** al più recente (variante di 2022.01.18/02.17 con ordine invertito).
 - **Teoria:** §5.5 SA_SIGINFO/siginfo_t; sincronizzazione mittente/ricevente con segnali.
-

4. Checklist operativa per il giorno della prova

1. **Es.0** subito: `chmod go-rx ~` + dir in `/public` a 700.
2. Identifica l'**archetipo** dell'Es.1: albero (`lsstat` +ricorsione), processi (`fork` / `exec` / `wait`), o eventi (segnali/pipe/timer/inotify).
3. Scrivi prima lo **scheletro** (§0) e compila spesso (`gcc -Wall`).
4. Per i link ricordati: `lsstat` **per vedere il link**, `stat` per il bersaglio, `readlink` per il testo del target, `realpath` per il path assoluto, `st_ino` + `st_dev` per riconoscere gli hard link.
5. Gestisci **sempre** gli errori con `perror` / `errno` (§2.9) e chiudi i fd.
6. Per i comandi: **mai** `system` / `popen` / `sh -c` — solo `fork` + `exec` + `wait` .
7. L'**Es.3** in bash/Python di solito è la stessa logica dell'Es.1: pensa a `find` , `stat` , `sort` , `sha1sum` , `grep` , e in Python a `os.walk` , `os.lstat` , `os.readlink` , `subprocess` .
8. Tieni a mente cosa **non** è nei due PDF (inotify, timerfd, pidfd, dlopen, getopt, realpath, sigqueue): per questi apri il `man` .